

2026 《人工智能导论》大作业

任务名称： 融合BERT分类与大语言模型解释的
谣言检测方法

完成组号： 9

小组人员： 葛庆晔 汪启瑞 马艺瑄 商志超

完成时间： 2026.6.15

1. 任务目标

基于谣言检测数据集，构建一个检测模型。该模型可以对数据集中的推文进行谣言检测，同时输出判断依据。要求：

- 输出 1：检测的输出是 2 分类（0 代表非谣言、1 代表谣言），分类准确率越高越好；
- 输出 2：判断依据的输出是一段文字，解释判断的依据；
- 模型具有一定的泛化能力；
- 有合理的运行时间。

2. 具体内容

（1）实施方案

本系统采用“BERT 分类 + LLM 解释”的复合架构：1. 检测模块：使用 BERT-base-uncased 预训练模型进行微调，完成谣言二分类任务。输入推文文本，输出 0（非谣言）或 1（谣言）。2. 解释模块：调用上海交通大学提供的 DeepSeek-V3.2 大语言模型接口，将原始文本和预测标签拼接成 Prompt，生成 50-80 字的中文判断依据。3. 容错机制：当 API 调用失败时，自动切换至本地规则模板，基于关键词和事件上下文生成基础解释。系统整体流程：输入文本 → BERT 分类 → 获取标签 → 调用 LLM 生成解释 → 输出（标签 + 解释）。

（2）核心代码分析

BERT 分类模型核心代码（BERTmodel.py）：

模型使用 Hugging Face Transformers 库加载预训练 BERT，并在下游添加二分类头。关键训练配置如下：

- 预训练模型：bert-base-uncased
- 最大序列长度：128
- 批大小：32
- 学习率：2e-5

– 训练轮数：10

解释生成核心代码（explanation_service.py）：

核心训练代码：

```
from transformers import BertTokenizer,
BertForSequenceClassification

from torch.utils.data import DataLoader

import torch


# 加载预训练模型和分词器

model = BertForSequenceClassification.from_pretrained('bert-base-
uncased', num_labels=2)

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')


# 数据编码

def encode(texts):

    return tokenizer(texts, padding=True, truncation=True,
max_length=128, return_tensors='pt')


# 训练循环

optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

for epoch in range(10):

    for batch in train_dataloader:

        outputs = model(**batch)
```

```
loss = outputs.loss
```

```
loss.backward()
```

```
optimizer.step()
```

解释生成核心代码（explanation_service.py）

调用交大 LLM API 生成判断依据，并实现本地规则作为备用方案：

```
from openai import OpenAI
```

```
# 调用交大 LLM API
```

```
def generate_explanation_by_sjtu_llm(text: str, pred_label: int) -> str:
```

```
    client = OpenAI(api_key=SJTU_API_KEY,  
base_url="https://models.sjtu.edu.cn/api/v1")
```

```
    response = client.chat.completions.create(
```

```
        model="deepseek-chat",
```

```
        messages=[
```

```
            {"role": "system", "content": "你是可解释性专家，必须认同分类模型的预  
测结果..."},
```

```
            {"role": "user", "content": f"文本：{text}\n预测标签：{pred_label}"}
```

```
        ],
```

```
        temperature=0.3,
```

```
        max_tokens=256
```

```
    )
```

```
    return response.choices[0].message.content.strip()
```

```
# 本地规则备用函数

def get_backup_explanation(text: str, pred_label: int) -> str:

    if pred_label == 1:

        return "该文本包含未经官方证实的表述，具有谣言特征。"

    else:

        return "该文本为客观事实陈述，无明显谣言特征。"
```

当 API 调用失败时，系统会降级使用本地规则模板，确保模型在任何环境下都能输出判断依据。

(3) 检测结果分析（正确率等）

一、模型性能汇总表

指标	值
准确率 (Accuracy)	85.29%
精确率 (Precision)	81.87%
召回率 (Recall)	85.14%
F1 分数	83.47%

二、结果分析

模型在验证集上取得了 85.29% 的准确率，表明模型对大多数样本能够正确分类。精确率为 81.87%，召回率为 85.14%，F1 分数为 83.47%。精确率略低于召回率，说明模型存在少量将非谣言误判为谣言的情况，但整体性能良好，能够满足谣言检测任务的基本要求。三项指标均超过 80%，证明 BERT 模型在该数据集上具有较好的分类能力和泛化能力。

(4) 判断依据的分析（可解释性等）

4.1 判断依据生成方法

本模型采用交大 DeepSeek-V3.2 大语言模型接口生成判断依据。具体流程为：

1. 将原始推文和模型预测的标签（0/1）拼接成 Prompt
2. 调用交大 LLM API，要求模型从文本表面特征（句式、情感倾向、信源提及方式）角度解释判定原因
3. 生成 50-80 字的中文解释，确保逻辑自洽且紧扣文本内容

同时，为应对 API 调用失败的情况，实现了本地规则备用函数，基于关键词和事件上下文生成基础解释。

4.2 判断依据示例

以下从验证集中选取了 4 个典型样本，展示模型输出的判断依据：

序号	推文内容	预测标签	判断依据
1	BREAKING: #Ferguson police chief just announced that officer Darren Wilson shot the unarmed teen, Michael Brown.	1 (谣言)	该推文使用“BREAKING”这一典型突发流言论头条词，且单方面声称“警察局长宣布”而缺乏多源官方背书，模型判定其为信息模糊的潜在谣言。
2	Milan have stated that the reports about Essien having Ebola are completely false.	1 (谣言)	该推文虽声称米兰官方辟谣，但仅以“have stated”这一模糊转述方式呈现，缺乏直接官方声明或权威链接，符合谣言模型对单方面非权威信息的抓取特征。
3	Lufthansa and Germanwings have established a telephone hotline. The toll-free 00800 11 33 55 77 number is available to all the families...	0 (非谣言)	该推文提供了具体热线号码及明确服务对象，语调客观且事实要素清晰，符合真实信息特征。
4	Our thoughts are with the family and friends of Cpl. Nathan Cirillo and all those affected by the events in Ottawa today. #OttawaStrong	0 (非谣言)	推文使用客观悼念用语，配合呼吁“Stay safe”及机构化标签#PrayForOttawa，语调冷静、事实指向明确，符合真实信息在情感表达和事实要素上的清晰特征。

4.3 可解释性评价

优点：解释与预测标签一致，能够指出具体文本特征（如“BREAKING”“单方面声称”），生成速度快

不足：少数情况下解释表述与标签的语义映射存在偏差；对隐含讽刺或复杂句式解释粒度较粗

3. 工作总结

（1）收获、心得

通过本次大作业，我们掌握了BERT预训练模型在文本分类任务上的微调方法，理解了Transformer架构的实际应用原理，学会了调用大语言模型API增强模型可解释性，体会了“分类+生成”复合模型的设计思路。同时，我们熟悉了GitHub团队协作开发流程，锻炼了跨角色沟通和接口对齐能力，提升了代码实现和问题调试的综合能力。

（2）遇到问题及解决思路

训练时遇到显存不足，通过将batch_size从32减小到16并启用梯度累积解决；模型对讽刺、质疑类文本识别准确率低，通过优化prompt引导LLM关注语气特征改善；LLM API偶发超时，增加重试机制（最多5次）

并实现本地规则模板作为降级方案；团队成员代码接口不统一，提前约定输入输出格式并制定接口文档解决；预测结果与解释存在语义偏差，通过加强system prompt中的约束指令修正。

4. 课程建议

建议增加中期检查节点避免期末集中赶工，提供更完整的可解释性参考示例，优化LLM API服务稳定性，并对“判断依据的可解释性”评分给出更细化的标准（如解释长度、与标签一致性、是否指出具体文本特征等）。